Maestro - AI-first higher education

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Promise basics and states

Today

Welcome to your next lesson: Promise basics and states.

I'm preparing your lesson based on what you've learned so far, what's next on your path, and the best way for you to build your skills.

Step by step, you’re moving forward.

Let’s get a feel for Promises with this quick video 📽

Watch it once — no need to catch every word, just get the overall idea of “pending / fulfilled / rejected”.

When you’re done, tell me in your own words:

If you request a ride in an app (Uber/Lyft style) and the car is still on its way,

would you say that ride request is finished or still in progress? 🚗

Still in progress

Exactly — still in progress ☑

In Promise terms, that ride request is pending.

Pending = the request has been made, but the result hasn’t arrived yet.

Let’s stick with the ride app:

You request a ride → pending
Later, either:
The driver picks you up → success
Or the request fails/cancels → failure

Both of those end states mean the ride is no longer “in progress”.

I’ll name those next. Ready for the next small step?

Yes, let's continue

Nice 🔥

Staying with the ride app:

Ride is pending while it’s on the way
When something finally happens, there are two options:

The driver arrives and you get picked up → the request succeeded
The driver never comes / gets cancelled → the request failed

In Promise language:

fulfilled = succeeded (ride completed)
rejected = failed (ride could not happen)
One more word:

settled = the Promise is no longer pending (it’s either fulfilled or rejected)

So:

pending → not settled
fulfilled or rejected → settled ☑

Does it make sense that both “success” and “failure” count as settled, because in both cases it’s over?

Yes, that makes sense

What do you think? 👇

In the ride app analogy, which of these situations is settled?

A.
You requested a ride and the car is still on the way.

B.
The driver arrived and picked you up.

C.
The app is still “searching for drivers”… maybe it will find one soon.

Nice job, Ron 🎯 You picked exactly the right one.

“The driver arrived and picked you up” is settled because the story is over — the request finished successfully → in Promise terms, fulfilled.
“Car is still on the way” or “searching for drivers” are both pending — still in progress, not settled yet.
Next tiny step: I want to connect this to the sealed envelope idea 📩

Imagine the ride app gives you a sealed envelope when you request a ride:

While the ride is pending, the envelope is sealed and you can't see inside.
When the ride is fulfilled or rejected, the envelope now contains the final result (success or failure) — but it was still sealed the whole time until that point.

A Promise is like that sealed envelope: it holds the future result, but you can't open/read it while it's pending.

Does that picture of a sealed envelope holding the future result make sense so far?

Yes, that makes sense

Awesome 👀

You've got:

pending = still in progress
fulfilled / rejected = finished, so settled
Promise = sealed envelope holding a future result you can't open yet
Now let's use that understanding with a few quick ride scenarios 🚌

I'll give you one at a time, and you just tell me which state it is in Promise words.

First one:

You request a ride. The app instantly shows: "Driver found, arriving in 3 minutes," and you see the car moving toward you on the map.

In Promise terms, is that pending, fulfilled, or rejected?

pending

Exactly right ☑

Ride found, on the way, not finished yet → pending Promise.

Next scenario:

You requested a ride. The driver arrived, picked you up, and you're already at your destination.

In Promise terms, is that fulfilled, rejected, or still pending?

I think it's fulfilled

Perfect 🎯

Reached destination successfully → the operation succeeded → fulfilled
And because it's no longer in progress, it's also settled.

Last one in this mini-set:

You request a ride. After a while, the app says: “No drivers available. Please try again later.”

In Promise terms, is that rejected or fulfilled?

I think it’s rejected

Exactly 💯

“No drivers available” = the ride request failed → in Promise terms: rejected
It’s also settled, because it’s definitely over (even though it failed).

So now you’ve mapped all three:

pending → still in progress
fulfilled → finished successfully (settled)
rejected → finished with failure (settled)
Next, I want to connect this to actual JavaScript behavior with a tiny example Promise and fetch. We’ll keep it super small.

Before we touch code: are you okay to switch over to the code editor for a short Promise experiment?

Yes, let's switch

Great 🚲 let’s keep this first code step very light.

In the code editor, type and run this:

Javascript
```
const promise = fetch("https://httpbin.org/delay/3");
console.log(promise);
```
This makes a request that waits ~3 seconds before responding.

Step by step what’s happening:

fetch(...) starts the request and immediately gives you a Promise (your sealed envelope).
console.log(promise) shows something like Promise {} in the console.

You won’t see the response data yet — just the Promise object itself.

Go ahead and run that in the editor, then tell me:

did you see something like Promise {} (or similar) printed?

yes

Nice, that’s exactly what I wanted 😄

Important point:

Seeing Promise {} does not mean “empty” or “broken”.
It just means: “Here is the Promise object (the sealed envelope). The console isn’t showing you what’s inside.”

Inside that Promise, the request is:

pending during the 3□second delay
then eventually becomes fulfilled (if it succeeds) or rejected (if it fails)

But the console log can’t show you that inner state directly.

Quick check, no trick here:

Does it make sense that when you log promise right after fetch(...), the Promise is still pending (the request is in progress), even though you already have the Promise object?

Yes, that makes sense

Great 👍

Let’s now connect the sealed envelope idea to when you can finally see the actual response content.

Right now you have:

Javascript
```
const promise = fetch("https://httpbin.org/delay/3");
console.log(promise);
```
Key idea:

promise is the envelope.
The actual response (the “letter” inside) becomes available only after the Promise is fulfilled (after the ~3 second delay).

You cannot just do:

Javascript
```
console.log(promise.body); // or similar
```
and expect the real data immediately, because while it’s pending, the envelope isn’t “openable”.

In JavaScript, we usually use .then(...) to say “when the Promise is fulfilled, then use the response”. I won’t go deep into the mechanics, but I’ll show you a tiny example next so you can see the actual response after the delay.

Does the timing idea make sense:

you get the Promise object right away, but the response content only exists later, when the Promise moves from pending to fulfilled?

Yes, that makes sense

Nice, you’re tracking this really well 🔥

Let’s very quickly contrast this with how Python often feels, then we’ll look at actual response content in JS.

In Python, a lot of code is blocking by default:

Python
```
response = requests.get("https://example.com")
print("Got response")
```
Here, line 1 usually waits until the response is ready. By the time line 2 runs, the “envelope” is already open — response holds the real data.

In JavaScript with Promises:

Javascript
const promise = fetch("https://httpbin.org/delay/3");
console.log("After fetch");
JS does not wait for the response before moving on.
promise is that sealed envelope — still pending when console.log("After fetch") runs.

So JS gives you a Promise object right away, and the actual response value only arrives later.

Next step: I want to show you a tiny .then(...) example just so you can see the real response after a few seconds (no need to deeply learn .then yet).

Ready for that small code step?

Yes, show me

Cool, let’s keep this super focused

In the code editor, replace your old code with this and run it:

Javascript
fetch("https://httpbin.org/delay/3").then((response) => {
  console.log("Got response object after delay:");
  console.log(response);
});
Step by step (high level only):

fetch(...) starts the request and gives a Promise (sealed envelope).
.then((response) => { ... }) means: “When the Promise is fulfilled, run this function with the response.”
After ~3 seconds, you should see the response object logged.

You don’t need to worry about how .then works internally yet — just notice that the real response only shows up after the delay.

Run that, watch the timing, and then tell me:

did console.log("Got response object after delay:") appear only after a few seconds?

Yes, it appeared after the delay

Perfect, that’s exactly what should happen

Let’s tie this together in plain language:

Right after fetch(...) runs → the Promise is pending (ride on the way, envelope sealed).
About 3 seconds later → the Promise becomes fulfilled (server replied, ride “arrived”).
Only then does the function inside .then((response) => { ... }) run, and you finally see the response object.
That’s the moment you can start working with the actual content of the response.

So:

You can access the real response content only after the Promise is fulfilled (settled successfully), not while it’s pending.

Quick understanding check (one concept, no tricks):

Imagine this timeline:

const p = fetch("https://httpbin.org/delay/3"); runs.
Immediately after, you log p.
Three seconds later, the server responds and the .then(...) callback runs.

At step 2, when you log p, which is true?

A) You already have the full response content.
B) You only have the Promise (sealed envelope), still pending.

You can just answer with A or B.

B

Exactly, B ☑

At step 2 you only have the Promise (sealed envelope), still pending.

The real response content only exists later, when the Promise is fulfilled and the .then(...) code runs.

Let’s do one last conceptual check on states and then we’ll be basically done with this lesson’s ideas:

Suppose you call:

Javascript
```
fetch("https://httpbin.org/delay/3").then((response) => {
  console.log("Done!");
});
```
Right after "Done!" is logged, which of these is true in Promise terms:

A) The Promise is fulfilled and settled.
B) The Promise is still pending.

Just answer A or B.

A

Exactly, A ☑

Right after "Done!" logs, the Promise has:

finished successfully → fulfilled
is no longer in progress → settled

So you’ve now nailed:

The three states: pending, fulfilled, rejected
settled = fulfilled or rejected (not pending)
Promise as a sealed envelope holding a future result
You only see/use the real response after the Promise is fulfilled (like in the .then(...) example)
You’ve hit all the key ideas for this lesson 🎯

Do you feel ready to wrap up this lesson and move on to the next one, or do you want one more quick scenario check before we finish?

Ready to move on

Nice work on this one, Ron 🎯

Here’s what you’re taking with you from this lesson:

A Promise moves through states:
pending → still in progress
fulfilled → finished successfully
rejected → finished with failure
settled = fulfilled or rejected (anything that’s not pending)
The ride app picture:
Searching / driver on the way → pending
Ride completed → fulfilled (settled)
No drivers / cancelled → rejected (settled)
The sealed envelope analogy:
The Promise is the envelope holding a future result.
While it’s pending, you can’t “open” it.
Once it’s fulfilled/rejected, the result inside is final.
With fetch("https://httpbin.org/delay/3"):
You get a Promise right away (pending).
A few seconds later, it becomes fulfilled and the .then((response) => { ... }) code finally sees the response object.
You can only access real response content after the Promise is fulfilled.

You’ve got a solid mental model now — that will make the next async tools (then, async/await, etc.) feel much more natural later on 🚀

When you’re ready, go ahead and jump into the next lesson in the Learn tab. If anything about Promises feels fuzzy later, just message me and we’ll tighten it up.

```
fetch("https://httpbin.org/delay/3").then((response) => {
  console.log("Got response object after delay:");
  console.log(response);
});
```